

ESTRUCTURAS DE DATOS - 2026-2 Implementación y Análisis de Complejidad de Listas, Pilas y Colas en Java

Estudiante: Marco Antonio García Villasmil Código: 1094050756 Profesor: David Herrera Monitora: Angela Camila Siabato Londoño

Repositorio: <https://github.com/marcogarciavillasmil/lab-listas-pilas-colas>

1. Objetivo del Taller

Acá tocaba armar en Java la estructura List con listas enlazadas, en sus cuatro variantes: simple sin cola, simple con cola, doble sin cola y doble con cola. También había que hacer Stack y Queue con arreglo dinámico (a la cola le tocaba ser un buffer circular para que dequeue no quede en $O(n)$). Con todo compilando y funcionando, lo que realmente importa del taller es medir cuánto se demora cada método según el tamaño de la entrada, comparar eso contra lo que predice la teoría en notación Big-O, y sacar conclusiones de cuál implementación conviene según el caso.

2. Implementación

List (Listas Enlazadas)

Para esta parte armé una interfaz común, MyList, con los 8 métodos que pide el enunciado: pushFront, pushBack, popFront, popBack, find, erase, addBefore y addAfter (también agregué topFront y topBack, que uso más adelante en la sección 7). Las 4 variantes de abajo implementan exactamente lo mismo, solo que manejan los punteros de forma distinta por dentro.

- **SinglyLinkedListNoTail:** La versión más simple de las cuatro, solo guarda un puntero a head. Cualquier operación que necesite llegar al final (pushBack, popBack, topBack) o encontrar el nodo anterior a otro (erase, addBefore) termina recorriendo la lista completa, porque no hay una forma más corta de llegar hasta ahí.
- **SinglyLinkedListWithTail:** Acá agregué un puntero tail, que resuelve pushBack y topBack de una (ya no toca recorrer), pero popBack sigue quedando mal: sin puntero al nodo anterior, para actualizar quién queda como nuevo tail toca recorrer igual desde el head.
- **DoublyLinkedListNoTail:** Cada nodo tiene ahora un puntero prev además del next, así que dado cualquier nodo ya se puede hacer erase, addBefore y addAfter en $O(1)$. Lo que queda pendiente: al no tener tail, llegar hasta el último nodo (pushBack, popBack, topBack) sigue costando $O(n)$.
- **DoublyLinkedListWithTail:** Esta junta las dos cosas (head, tail, prev y next en cada nodo) y es la única que queda en $O(1)$ en casi todo. La excepción es find, que al no haber indexación por posición no tiene otra opción que recorrer.

Algo que decidí desde el principio: find() no devuelve un booleano ni un índice, devuelve directamente la referencia al Node. La idea es no buscar el elemento dos veces: si solo dijera "sí está" o "no está", borrarlo o insertar algo al lado después tocaría recorrer la lista otra vez para ubicarlo, otro $O(n)$ de paso. Al devolver el Node, ese segundo paso queda en pura manipulación de punteros. Por eso en la tabla de 4.1 las complejidades de erase, addBefore y addAfter asumen que ya se tiene el Node en la mano (el que devolvió find()); no incluyen el costo de buscarlo, que en las 4 variantes sigue siendo $O(n)$ porque ninguna tiene indexación directa.

MyStack y MyQueue

Para el Stack (ArrayStack) usé un arreglo dinámico normal: un arreglo interno que se duplica cada vez que se llena, lo mismo que hace ArrayList por dentro. Como push y pop operan siempre sobre el tope (la última posición ocupada), las dos quedan en $O(1)$ amortizado sin necesidad de nada más elaborado.

Para el Queue (ArrayQueue) la primera versión que hice era el arreglo "de la forma obvia": enqueue metía al final sin drama, pero dequeue tenía que sacar la posición 0 y correr todos los elementos un puesto hacia atrás, dejando esa operación en $O(n)$ (con $n=1\,000\,000$ esa primera versión llegaba a unos 8594 μs por dequeue, medido). Para arreglar eso reescribí ArrayQueue como un buffer circular: en vez de siempre insertar/sacar en las posiciones 0 y size-1, guardo un índice head y calculo las posiciones módulo la capacidad del arreglo. Así dequeue solo mueve el índice head una posición (sin correr nada) y queda en $O(1)$, igual que enqueue y front. Al redimensionar (grow), copio los elementos respetando su orden lógico desde head y reinicio head en 0 sobre el arreglo nuevo.

Para el crecimiento del arreglo (tanto en Stack como en Queue) usé la estrategia típica de amortización: cuando se llena, se duplica la capacidad (capacidad $\ast 2$). Puede que una inserción puntual salga cara ($O(n)$ por el redimensionamiento), pero si se ve el costo total de n inserciones, sigue siendo $O(n)$ en total, es decir $O(1)$ amortizado por inserción.

3. Metodología de Medición

Para medir usé System.nanoTime() y reporté todo en microsegundos, porque con milisegundos y listas chiquitas todo se redondeaba a 0 y no se notaba la diferencia entre $O(1)$ y $O(n)$. La mecánica para cada tamaño n : armar la estructura con esos n elementos usando un método $O(1)$ (para no distorsionar lo que quiero medir), y luego repetir el método varias veces y sacar el promedio. En push/pop deshago cada operación justo después de medirla, para que el tamaño no cambie a mitad del experimento. En los métodos de búsqueda (find, erase, addBefore, addAfter, delete) el objetivo se elige al azar, con semilla fija en 42 para que sea reproducible. En erase/addBefore/addAfter, si el índice al azar no corresponde a ningún nodo (puede pasar porque el mismo valor se repite o ya se borró antes en la misma corrida), esa repetición se descarta del promedio en vez de contarla con tiempo cero, para no sesgar el resultado a la baja. Los tamaños fueron $n = 10, 100, 1000, 10\,000, 100\,000$ y $1\,000\,000$; al principio quería llegar hasta 10^8 como sugiere el enunciado, pero una sola corrida de pushBack en SinglyNoTail con $n=10^6$ ya se demoraba varios minutos -con 10^8 el laboratorio no se terminaba nunca-, así que me quedé en 10^6 : en las gráficas ya se ve clarísima la tendencia (recta que sube en log-log para $O(n)$, línea plana para $O(1)$), y subir más solo iba a aumentar el tiempo de espera, no a cambiar la conclusión.

Antes de empezar a medir, cada clase de benchmark corre un calentamiento de la JVM: ejecuta miles de operaciones de cada método sobre una estructura chica, sin registrar nada, para que el JIT alcance a compilar esos métodos a código nativo antes de la medición real. Sin esto, las primeras repeticiones de cada corrida salen artificialmente lentas y contaminan el promedio, sobre todo en los tamaños con pocas repeticiones.

Ojo que no usé la misma cantidad de repeticiones en todos los tamaños. Para pushFront/pushBack/popFront/popBack/topFront/topBack/find usé 300 repeticiones en $n=10, 100$ y 1000 , bajando a 150 en $n=10\,000$, 50 en $n=100\,000$ y apenas 15 en $n=1\,000\,000$ (si no, la corrida se demoraba horas en los métodos $O(n)$ con n grande). Para erase/addBefore/addAfter/delete, ya más costosos de por sí, usé

solo 5 repeticiones en $n=10$ y $n=100$. Esto importa porque con pocas repeticiones el promedio queda más expuesto al ruido: una pausa del recolector de basura o del sistema pesa más sobre el promedio.

4. Complejidad Teórica

4.1 List: Complejidad Teórica

método	SinglyNoTail	SinglyWithTail	DoublyNoTail	DoublyWithTail
pushFront	$O(1)$	$O(1)$	$O(1)$	$O(1)$
pushBack	$O(n)$	$O(1)$	$O(n)$	$O(1)$
popFront	$O(1)$	$O(1)$	$O(1)$	$O(1)$
popBack	$O(n)$	$O(n)$	$O(n)$	$O(1)$
topFront	$O(1)$	$O(1)$	$O(1)$	$O(1)$
topBack	$O(n)$	$O(1)$	$O(n)$	$O(1)$
find	$O(n)$	$O(n)$	$O(n)$	$O(n)$
erase	$O(n)$	$O(n)$	$O(1)$	$O(1)$
addBefore	$O(n)$	$O(n)$	$O(1)$	$O(1)$
addAfter	$O(1)$	$O(1)$	$O(1)$	$O(1)$

4.2 MyStack y MyQueue: Complejidad Teórica

operación	MyStack (arreglo dinámico)	MyQueue (arreglo circular)
push / enqueue	$O(1)$ amortizado	$O(1)$ amortizado
pop / dequeue	$O(1)$	$O(1)$
peek / front	$O(1)$	$O(1)$
delete(valor)	$O(n)$	$O(n)$

5. Hipótesis

Hipótesis sobre List

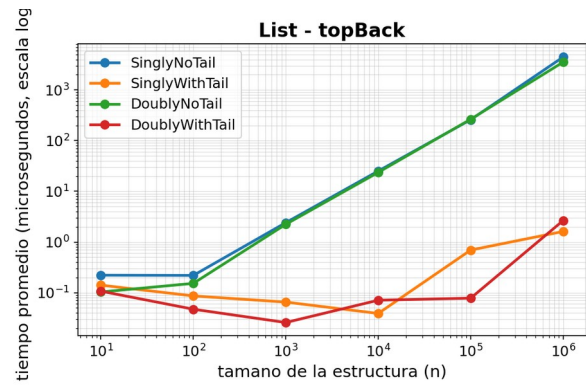
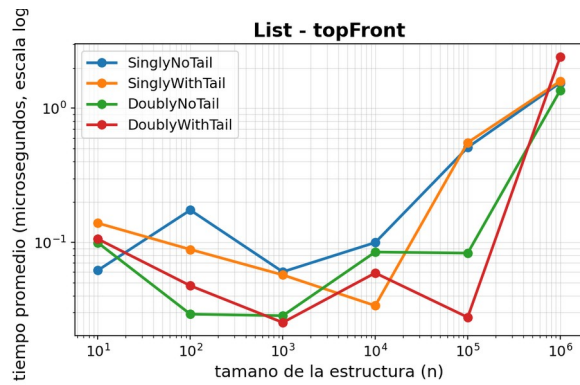
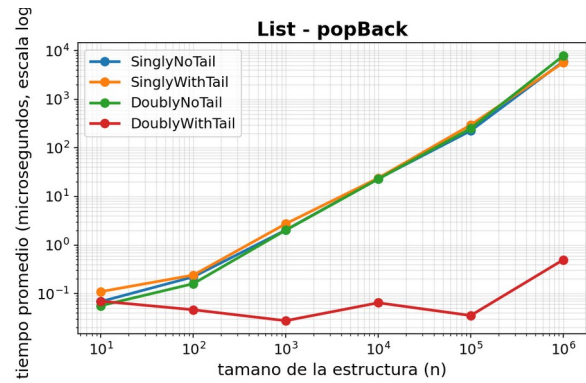
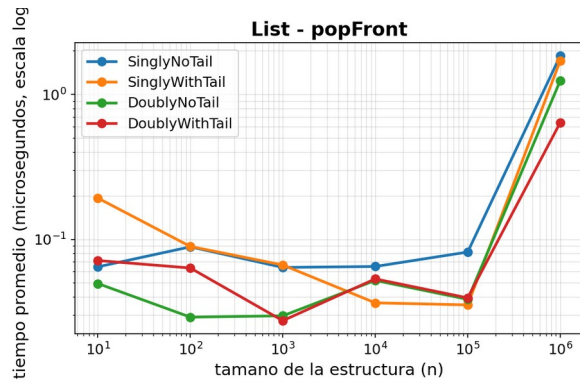
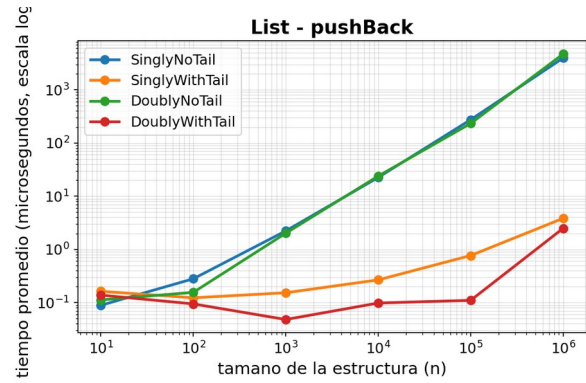
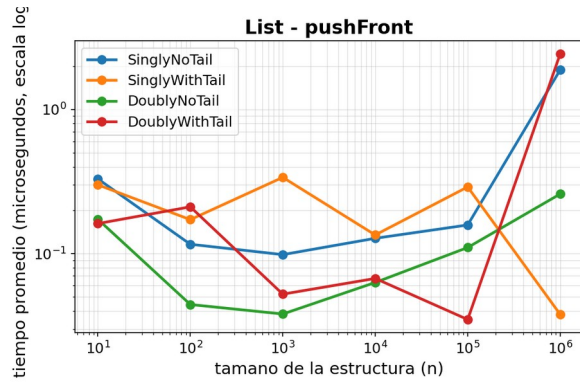
Antes de correr el benchmark, con solo mirar la tabla de arriba ya me imaginaba qué iba a pasar: pushFront, popFront, topFront y addAfter deberían mantenerse casi planos ($O(1)$) en las 4 variantes por igual; pushBack, popBack y topBack solo rápidos en las versiones con tail (o con prev, para popBack en DoublyWithTail), y creciendo en las demás; y find lento ($O(n)$) en las 4, porque ninguna variante indexa directo. Ahora toca ver qué tanto le atiné comparando esto con lo medido.

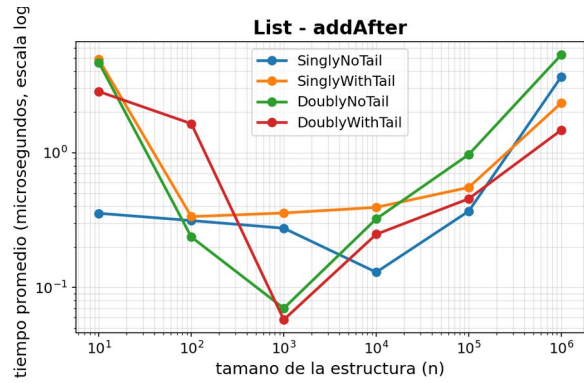
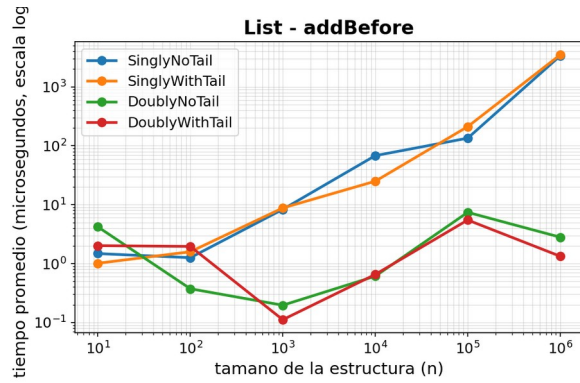
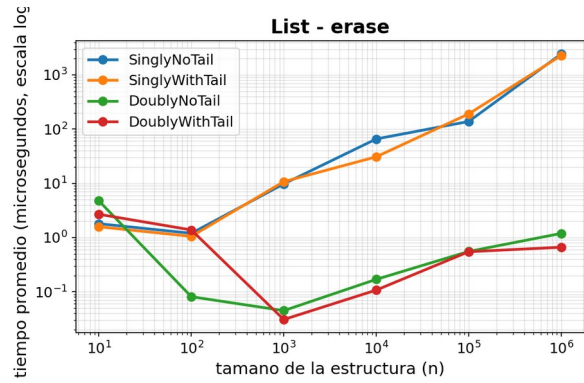
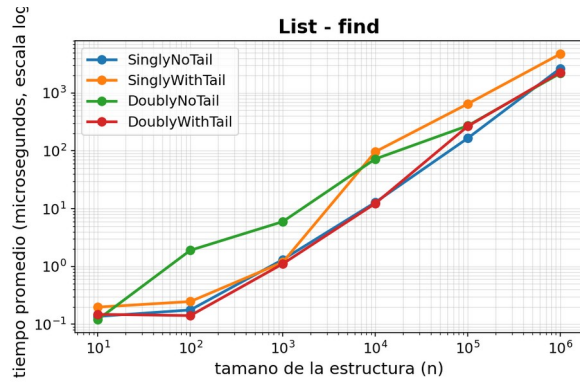
Hipótesis sobre MyStack y MyQueue

Acá la hipótesis era sencilla, porque diseñé las dos estructuras desde el principio para que todo lo básico quedara en $O(1)$: push/pop/peek en el Stack porque siempre operan sobre el tope, y enqueue/dequeue/front en el Queue porque el buffer circular evita correr el arreglo. Donde sí esperaba tiempos más largos y creciendo con n era en delete de las dos, porque primero toca buscar el valor ($O(n)$) y después cerrar el espacio corriendo el resto del arreglo.

6. Resultados

6.1 List: Resultados Empíricos por Método





SinglyLinkedListNoTail

n	10	100	1000	10000	100000	1000000
pushFront	0.33	0.12	0.10	0.13	0.16	1.88
pushBack	0.09	0.28	2.22	22.50	274.26	3951.83
popFront	0.06	0.09	0.06	0.06	0.08	1.84
popBack	0.07	0.22	2.03	23.14	224.62	5949.06
topFront	0.06	0.17	0.06	0.10	0.51	1.55
topBack	0.22	0.22	2.41	25.18	258.97	4464.61
find	0.14	0.18	1.31	12.81	164.93	2649.07
erase	1.80	1.20	9.76	65.78	138.18	2424.24
addBefore	1.48	1.26	8.32	67.96	134.64	3359.23
addAfter	0.35	0.31	0.28	0.13	0.37	3.64

SinglyLinkedListWithTail

n	10	100	1000	10000	100000	1000000
pushFront	0.30	0.17	0.34	0.13	0.29	0.04
pushBack	0.16	0.12	0.15	0.27	0.77	3.83
popFront	0.19	0.09	0.07	0.04	0.04	1.70
popBack	0.11	0.24	2.74	23.90	296.19	5641.30
topFront	0.14	0.09	0.06	0.03	0.55	1.59
topBack	0.14	0.09	0.07	0.04	0.69	1.62
find	0.20	0.25	1.18	96.10	647.33	4716.02
erase	1.58	1.05	10.74	30.79	190.62	2267.83
addBefore	1.01	1.59	8.78	24.98	211.13	3492.66
addAfter	4.91	0.34	0.36	0.39	0.55	2.33

DoublyLinkedListNoTail

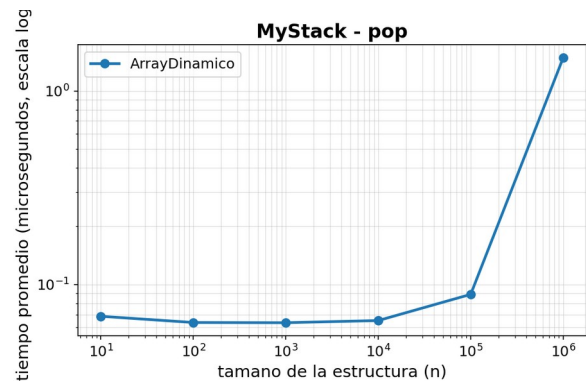
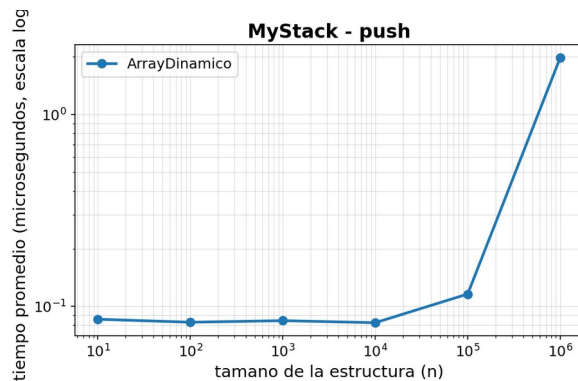
n	10	100	1000	10000	100000	1000000
pushFront	0.17	0.04	0.04	0.06	0.11	0.26
pushBack	0.11	0.16	2.01	23.88	233.02	4703.29
popFront	0.05	0.03	0.03	0.05	0.04	1.24
popBack	0.06	0.16	2.02	22.82	251.20	7813.95
topFront	0.10	0.03	0.03	0.08	0.08	1.36
topBack	0.10	0.15	2.23	23.64	264.53	3540.76
find	0.12	1.90	5.96	72.17	271.18	2182.28
erase	4.75	0.08	0.04	0.17	0.55	1.19
addBefore	4.20	0.37	0.20	0.61	7.41	2.80
addAfter	4.65	0.24	0.07	0.32	0.97	5.31

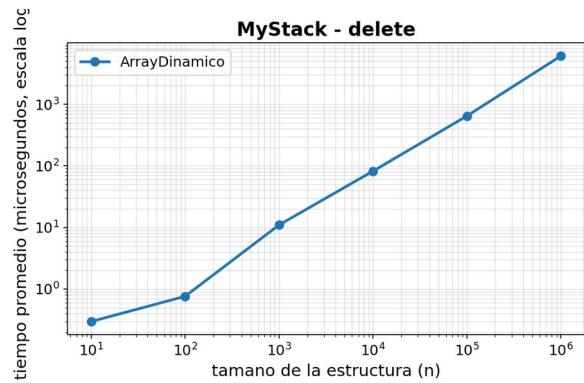
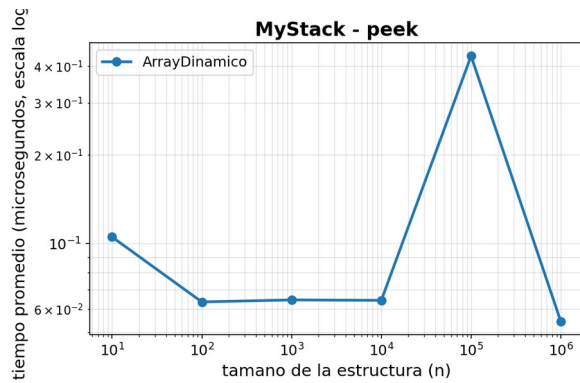
DoublyLinkedListWithTail

n	10	100	1000	10000	100000	1000000
pushFront	0.16	0.21	0.05	0.07	0.03	2.43
pushBack	0.14	0.09	0.05	0.10	0.11	2.49
popFront	0.07	0.06	0.03	0.05	0.04	0.64
popBack	0.07	0.05	0.03	0.06	0.04	0.49
topFront	0.11	0.05	0.03	0.06	0.03	2.41
topBack	0.11	0.05	0.03	0.07	0.08	2.65
find	0.15	0.14	1.11	12.28	265.09	2268.61
erase	2.69	1.38	0.03	0.11	0.55	0.66
addBefore	2.01	1.95	0.11	0.65	5.48	1.33
addAfter	2.84	1.64	0.06	0.25	0.45	1.47

Comparando esto con la hipótesis: en general sí se cumplió el patrón esperado, sobre todo en n grande (10^5 y 10^6), donde las curvas $O(n)$ (pushBack y topBack sin tail, popBack sin prev, find, erase/addBefore sin prev) se separan bastante de las que quedan planas. topFront en particular se mantiene por debajo de $1\text{-}2\ \mu\text{s}$ en las 4 variantes incluso en $n=1\ 000\ 000$, confirmando que no depende de tail ni de prev. Donde no coincidió tan limpio fue en algunos tamaños chicos e intermedios: varios métodos en teoría $O(1)$ muestran picos puntuales que no tienen que ver con la complejidad sino con ruido del sistema (aun con el calentamiento de JVM, una pausa del recolector de basura pesa más cuando lo que se está midiendo son solo unos pocos nanosegundos por operación).

6.2 MyStack: Resultados Empíricos

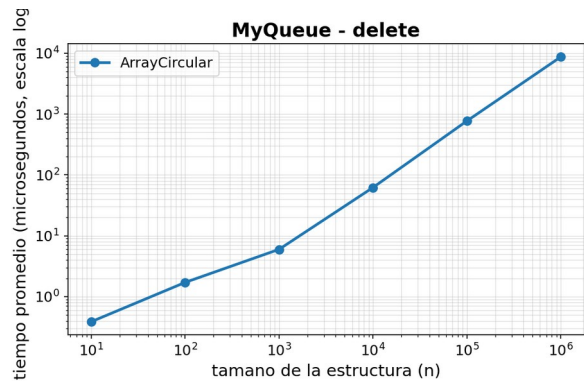
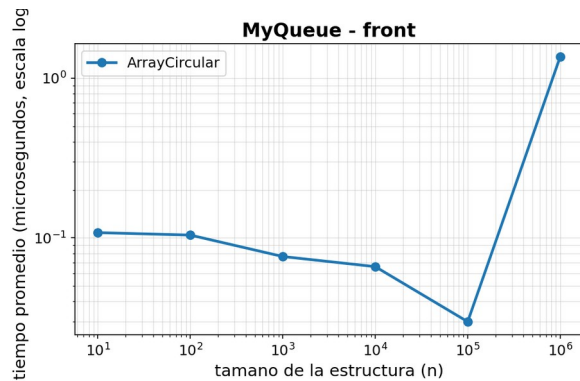
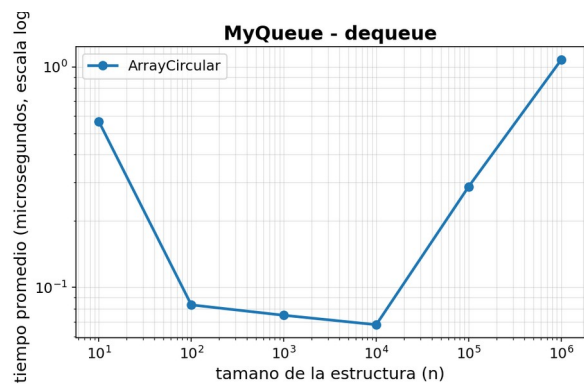
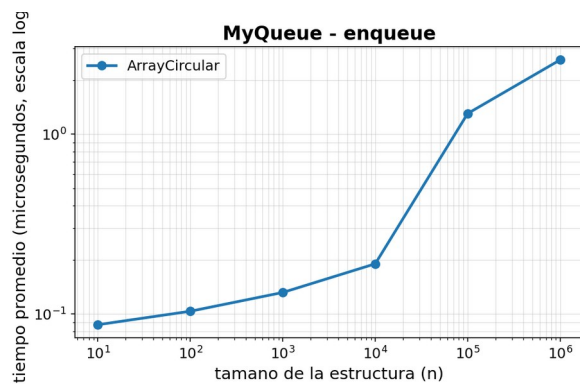




n	10	100	1000	10000	100000	1000000
push	0.09	0.08	0.08	0.08	0.12	1.97
pop	0.07	0.06	0.06	0.07	0.09	1.48
peek	0.11	0.06	0.06	0.06	0.43	0.05
delete	0.30	0.76	10.96	81.86	643.24	6047.72

Comparando con la hipótesis: push, pop y peek se mantienen bajos y sin tendencia de crecimiento en todos los tamaños (por ejemplo push pasa de 0.09 μ s en $n=10$ a apenas 1.97 μ s en $n=1\,000\,000$), como se esperaba de operaciones $O(1)$. delete sí crece claramente con n , llegando a 6047.72 μ s en $n=1\,000\,000$, coherente con el $O(n)$ de buscar más correr el arreglo.

6.3 MyQueue: Resultados Empíricos



n	10	100	1000	10000	100000	1000000
---	----	-----	------	-------	--------	---------

enqueue	0.09	0.10	0.13	0.19	1.30	2.59
dequeue	0.57	0.08	0.07	0.07	0.29	1.08
front	0.11	0.10	0.08	0.07	0.03	1.36
delete	0.39	1.71	5.98	62.07	769.37	8689.86

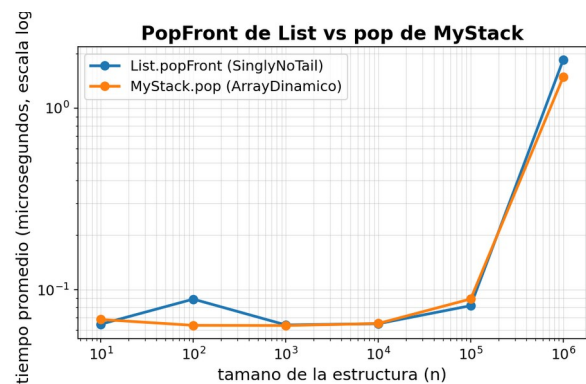
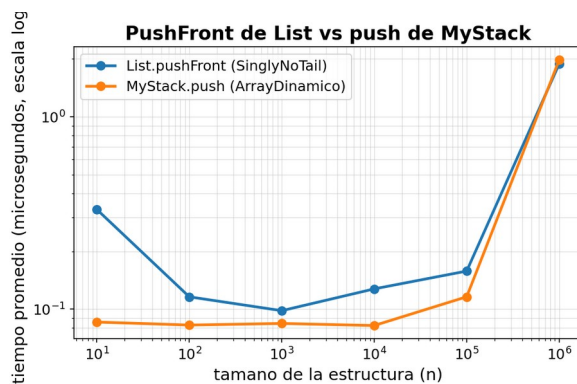
Acá se nota justo lo que buscaba al rehacer la cola como buffer circular: dequeue se mantiene en 1.08 μ s incluso en $n=1\,000\,000$, contra los cerca de 8594 μ s que daba la primera versión (la que corría todo el arreglo). enqueue y front también se quedan bajos en todos los tamaños. delete sigue siendo la excepción esperada: crece hasta 8689.86 μ s en $n=1\,000\,000$, porque buscar el valor y cerrar el espacio en el buffer circular sigue siendo $O(n)$, el buffer circular no ayuda con esto.

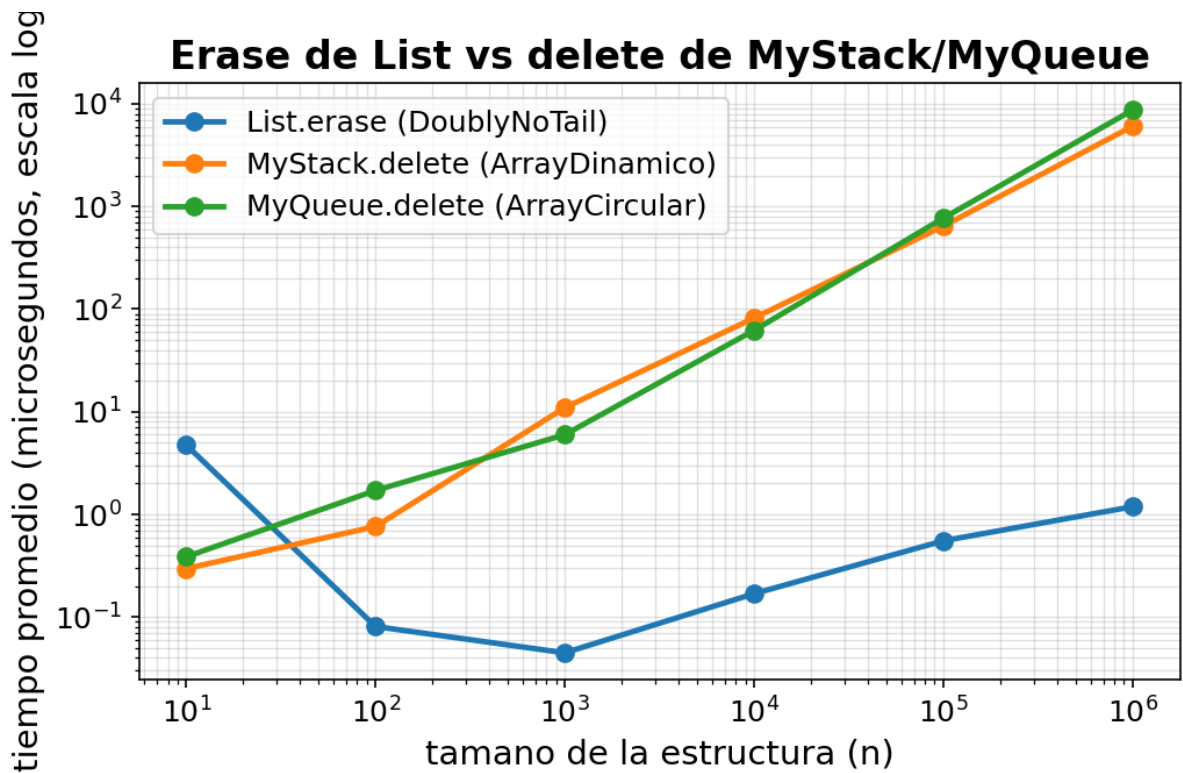
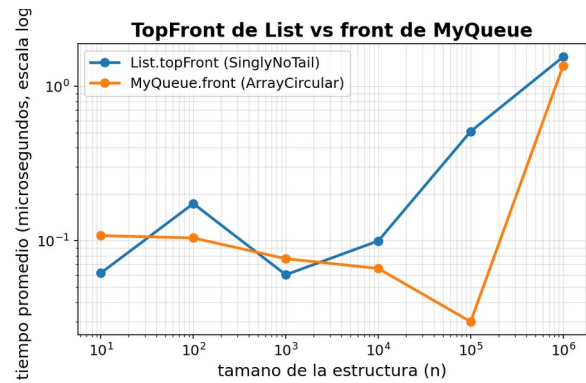
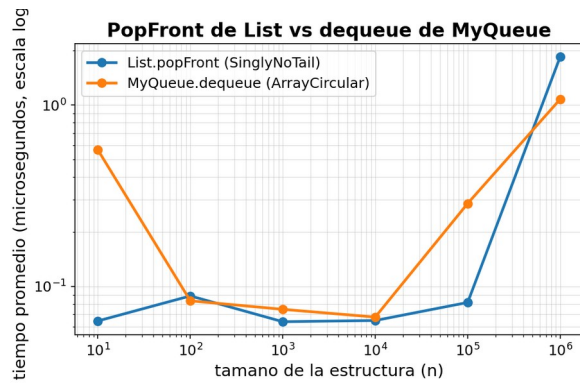
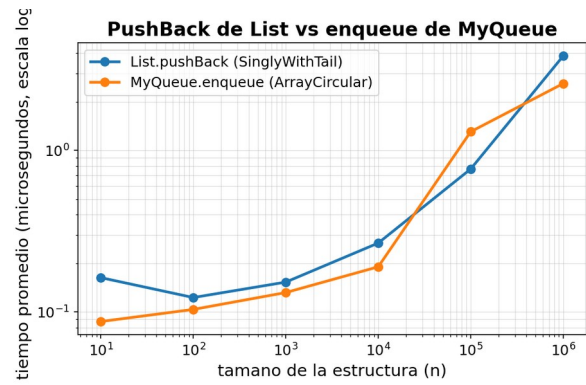
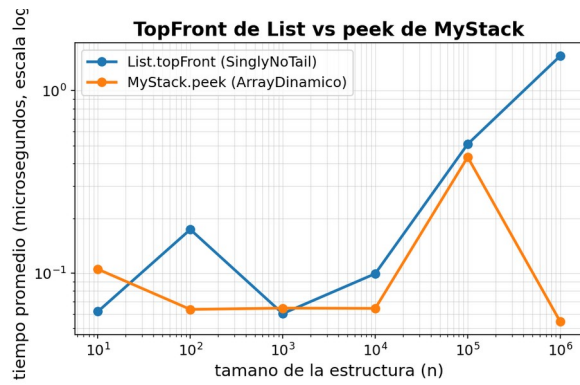
7. Comparación entre Implementaciones

Comparativa de Métodos Equivalentes (List vs MyStack/MyQueue)

El enunciado pide elegir, para cada método, la implementación de List que dé la complejidad más óptima -no tiene que ser siempre la misma variante-, y explicar por qué. Push/pop de MyStack equivalen a pushFront/popFront de List (ambos operan sobre un mismo extremo), mientras que enqueue/dequeue de MyQueue equivalen a pushBack/popFront (se inserta por un extremo y se saca por el otro). Con eso en mente, elegí así:

- **PushFront vs push, PopFront vs pop/dequeue, TopFront vs peek/front:** estos métodos son $O(1)$ por igual en las 4 variantes de List (topFront/pushFront/popFront solo leen o modifican head, no dependen de tail ni de prev), así que no tenía sentido usar la variante más pesada: elegí SinglyLinkedListNoTail, la más simple que ya da la misma complejidad óptima.
- **PushBack vs enqueue:** pushBack es $O(1)$ solo en las variantes con tail. Entre esas dos, elegí SinglyLinkedListWithTail en vez de DoublyLinkedListWithTail, porque el puntero prev de la versión doble no aporta nada extra para este método en particular.
- **Erase vs delete (Stack y Queue):** erase es $O(1)$ solo en las variantes Doubly (necesitan prev para no tener que recorrer buscando el nodo anterior). Entre esas dos elegí DoublyLinkedListNoTail, porque el puntero tail de la versión con cola no ayuda en nada para borrar un nodo interno.





- **Push/Pop/Peek/Enqueue/Dequeue/Front:** en las seis comparaciones las estructuras quedan prácticamente empatadas, todas $O(1)$. Esto es distinto a lo que pasaba antes de que la cola fuera circular:

con el dequeue viejo ($O(n)$) la comparación contra `List.popFront` hubiera mostrado una brecha enorme en n grande; ahora que `MyQueue` es un buffer circular, esa brecha desaparece.

- **Delete / Erase:** acá sí hay diferencia real: en eliminación por valor, `List` le gana a `Stack/Queue` basados en arreglo. Las tres son $O(n)$ para encontrar el valor, eso no cambia, pero en la lista enlazada, una vez encontrado el nodo, eliminarlo es $O(1)$; en un arreglo (circular o no) toca correr los elementos que quedan para cerrar el espacio.

8. Conclusiones

- **Listas Enlazadas:** ninguna implementación de `List` gana en todo. `SinglyLinkedListNoTail` es la más fácil de programar, pero no escala en las operaciones que tocan el final: `pushBack` pasa de $0.09\ \mu s$ en $n=10$ a $3951.83\ \mu s$ en $n=1\ 000\ 000$. Si lo que se busca es eficiencia $O(1)$ en ambos extremos (incluyendo `topBack`), no tiene vuelta de hoja: toca tener doble enlace y puntero a tail al mismo tiempo, o sea `DoublyLinkedListWithTail`.
- **Por qué la cola quedó como buffer circular:** la primera versión de `ArrayQueue` desplazaba todo el arreglo en cada dequeue, y eso se notaba muchísimo con n grande (cerca de $8594\ \mu s$ en $n=1\ 000\ 000$). Al reescribirla como buffer circular (índice head módulo la capacidad), dequeue bajó a $1.08\ \mu s$ en el mismo tamaño, casi igual que enqueue y front. Para mí fue lo que más se notó en todo el taller: una cola sobre arreglo sin hacerla circular no sirve con n grande.
- **Uso en escenarios reales:** una pila (`Stack`) es la estructura natural cuando el orden de deshacer importa: la pila de llamadas de cualquier programa, el `Ctrl+Z` de un editor, o evaluar expresiones matemáticas con paréntesis. Una cola (`Queue`) encaja mejor cuando hay que procesar las cosas en el mismo orden en que llegaron: una cola de impresión, los mensajes de un sistema de colas (`RabbitMQ`, `SQS`), o el recorrido BFS de un grafo. En ambos casos, si el número máximo de elementos es predecible, un arreglo (dinámico o circular) gasta menos memoria; si no se sabe de antemano o se necesita insertar/borrar en medio, una lista enlazada da más flexibilidad sin pagar el costo de mover memoria.
- **Arreglos vs Listas:** los arreglos siguen ganando en localidad de memoria y costo por elemento sobre extremos fijos, y con el buffer circular la cola ya no tiene ninguna desventaja de fondo frente a la lista en las operaciones básicas. La única diferencia real que queda es delete/erase: la lista se mantiene por debajo de un par de microsegundos incluso en $n=1\ 000\ 000$, mientras que delete en las estructuras de arreglo sube a varios miles de microsegundos, porque además de encontrar el valor ($O(n)$ en las tres) el arreglo tiene que correr los elementos para cerrar el espacio. Si lo que se necesita es insertar o eliminar seguido en medio de la estructura, ahí las listas enlazadas siguen siendo la mejor opción.

9. Uso de Inteligencia Artificial

En este trabajo usé inteligencia artificial, específicamente Claude (Anthropic), como apoyo para organizar el documento final y la estructura del repositorio en GitHub, y para revisar el repositorio contra el enunciado (detectar que faltaba el buffer circular en la cola, un bug en cómo promediaba ciertas mediciones, y que faltaban un par de comparativas de la parte 3). El código de las cuatro implementaciones de `List`, de `MyStack` y `MyQueue`, los benchmarks, las pruebas y el análisis/conclusiones los hice yo a mano. Las gráficas también las generé yo con Python (matplotlib), a partir de los datos de los benchmarks (los csv en `results/`); lo que hizo la IA fue acomodar las imágenes y tablas dentro del documento de Word y ayudar a redactar. En general calculo que el apoyo de la IA estuvo alrededor de un 35%, porque el análisis y los

procedimientos/requerimientos que pedía el taller los resolví yo. Todo el contenido generado con ayuda de IA fue revisado y editado por mí para que la información fuera precisa.